KATHMANDU UNIVERSITY

SCHOOL OF ENGINEERING

DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

MINI PROJECT REPORT



EEEG 320

DIGITAL CLOCK

by:

Aayush Adhikari (32191)

Saman Fuyal (32199)

Anup Shrestha (32241)

Supervisor: Prof. Dr. Ram Kaji Budhathoki

20th January 2026

ABSTRACT

This project presents the design and implementation of a digital clock using VHDL on the Nexys A7 (Artix-7) FPGA development board. The system displays time in hours, minutes, and seconds using the onboard seven-segment display. A 100 MHz clock provided by the FPGA is divided to generate a 1 Hz timing signal for accurate time counting. The clock operates in two modes: a normal mode, where time advances automatically, and a set mode, where the user can manually increment hours and minutes using push buttons. Button inputs are debounced to ensure reliable operation, and time values are converted from binary to BCD format for display purposes. The design is modular, consisting of clock generation, timekeeping, conversion, and display driver units, all integrated at the top level. The functionality of the digital clock was verified through simulation and hardware testing on the Nexys A7 board, demonstrating correct time counting, adjustment, and display behavior.

TABLE OF CONTENTS

ABSTRACT i

1. INTRODUCTION 1

2. OBJECTIVES 1

3. METHODOLOGY 1

3.1 Hardware Platform 1

3.2 System Architecture 1

3.3 System Operation and Design Flow 2

4. VHDL DESIGN CODE 4

4.1 Clock Enable Generator (clk_en_gen.vhd) 4

4.2 Debounce Module (debounce.vhd) 5

4.3 Timekeeper (timekeeper.vhd) 6

4.4 Binary-to-BCD Converter (bin_to_bcd2.vhd) 7

4.5 Seven-Segment Display Driver (sevenseg_mux8.vhd) 7

4.6 Top-Level Module (top_clock_nexys_a7.vhd) 9

5. CONSTRAINTS FILE (nexys_a7_clock.xdc) 11

6. RESULTS 13

6.1 Functional Verification 13

6.2 RTL Schematic 13

6.3 Hardware Output on Nexys A7 Board 14

6.4 Stopwatch-Style Testing 14

7. CONCLUSION 15

8. FUTURE IMPROVEMENTS 15

REFERENCES 16

List of Figures

Figure 1: Block Diagram 2
Figure 2: RTL Schematic 13
Figure 3: Current time in HH:MM:SS format 14
Figure 4: Stopwatch comparison test 14

1. INTRODUCTION

A digital clock is a fundamental digital system used to display time in hours, minutes, and seconds. Implementing a digital clock on an FPGA helps in understanding key digital design concepts such as clock division, counters, synchronous logic, debouncing of mechanical inputs, and multiplexed display driving.

This project presents the design and implementation of a digital clock using VHDL on the Nexys A7 (Artix-7) FPGA board. The system uses the onboard 100 MHz clock, push buttons, slide switches, and the seven-segment display to realize a functional clock capable of normal operation and manual time adjustment

2. OBJECTIVES

The objectives of this project are:

1. To design a real-time digital clock using VHDL.
2. To display time in HH:MM:SS format on the seven-segment display.
3. To allow manual increment of hours and minutes.

3. METHODOLOGY

3.1 Hardware Platform

- FPGA Board: Nexys A7-50T (Artix-7)
- Clock Source: 100 MHz onboard oscillator
- Inputs: SW0 – Set mode, BTNU – Hour increment, BTND – Minute increment, BTNC – Reset
- Output: 8-digit common-anode seven-segment display

3.2 System Architecture

The design is divided into the following VHDL modules:

1. Clock Enable Generator
2. Debounce Circuit
3. Timekeeper
4. Binary-to-BCD Converter
5. Seven-Segment Display Driver
6. Top-Level Integration

3.3 System Operation and Design Flow

The digital clock system operates by dividing the onboard FPGA clock, maintaining time through synchronous counters, and displaying the result using a multiplexed seven-segment display. The overall working of the system can be explained in the following steps.

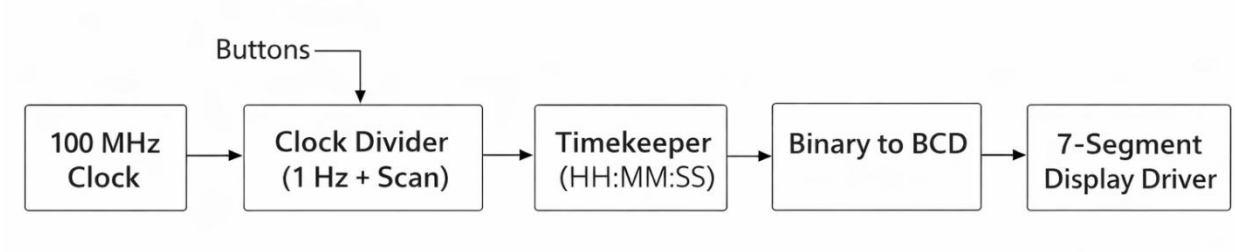


Figure 1: Block Diagram

Step 1: Clock Generation and Timing Control

The Nexys A7 FPGA provides a 100 MHz system clock. This clock is too fast for human-readable timekeeping, so it is first processed by a clock enable generator.

- The clock enable generator divides the 100 MHz clock to produce:
    - a 1 Hz enable signal for updating seconds, minutes, and hours
    - a high-frequency scan signal for multiplexing the seven-segment display

This ensures accurate time counting and flicker-free display operation.

Step 2: Button Input and Debouncing

Mechanical push buttons produce unwanted bouncing when pressed. To avoid multiple false triggers, each button is connected to a debounce module.

- The debounce circuit:
    - synchronizes the button input to the system clock
    - filters out noise and bouncing
    - generates a single clock pulse for each valid press

These clean pulses are used to increment hours and minutes during time-set mode.

Step 3: Timekeeping Logic

The core functionality of the system is handled by the timekeeper module.

- In normal mode (SW0 = OFF):
    - the 1 Hz clock enable increments the seconds counter
    - seconds roll over to minutes
    - minutes roll over to hours
    - hours roll over after 23
- In set mode (SW0 = ON):
    - normal time counting is paused
    - pressing BTNU increments the hour
    - pressing BTND increments the minute
    - seconds remain unchanged

This allows the user to manually adjust the time.

Step 4: Binary to BCD Conversion

The timekeeper stores time in binary format, which cannot be directly displayed on a seven-segment display. Therefore, a binary-to-BCD converter is used.

- Each time value (hour, minute, second) is split into:
    - tens digit
    - ones digit

These BCD digits are passed to the display driver.

Step 5: Seven-Segment Display Multiplexing

The Nexys A7 uses a common-anode multiplexed seven-segment display. Only one digit is activated at a time.

- The seven-segment driver:
    - cycles through all digits using the scan clock
    - selects the appropriate BCD digit
    - converts it into segment patterns
    - activates the corresponding anode

Because the scanning occurs rapidly, all digits appear illuminated simultaneously to the human eye.

Step 6: Top-Level Integration

The top-level module connects all sub-modules together:

- distributes the system clock and reset
- routes debounced button pulses to the timekeeper
- connects BCD outputs to the display driver
- ensures correct synchronization between all components

This modular design improves clarity, debugging, and scalability.

4. VHDL DESIGN CODE

4.1 Clock Enable Generator (`clk_en_gen.vhd`)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity clk_en_gen is
  generic (
    CLK_HZ  : natural := 100_000_000;
    SCAN_HZ : natural := 1000
  );
  port (
    clk        : in  std_logic;
    rst        : in  std_logic;
    tick_1hz   : out std_logic;
    tick_scan : out std_logic
  );
end entity;

architecture rtl of clk_en_gen is
  signal cnt1 : unsigned(26 downto 0) := (others => '0');
  signal cnt2 : unsigned(16 downto 0) := (others => '0');
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        cnt1 <= (others => '0');
        cnt2 <= (others => '0');
        tick_1hz <= '0';
        tick_scan <= '0';
      else
        tick_1hz <= '0';
        tick_scan <= '0';

        if cnt1 = CLK_HZ - 1 then
          cnt1 <= (others => '0');
          tick_1hz <= '1';
        else
```

```
          cnt1 <= cnt1 + 1;
        end if;

        if cnt2 = (CLK_HZ/SCAN_HZ) - 1 then
          cnt2 <= (others => '0');
          tick_scan <= '1';
        else
          cnt2 <= cnt2 + 1;
        end if;
      end if;
    end if;
  end process;
end architecture;
```

4.2 Debounce Module (`debounce.vhd`)

This module removes mechanical button bouncing and generates a single-clock pulse per press.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity debounce is
  generic (
    CLK_HZ      : natural := 100_000_000;
    DEBOUNCE_MS : natural := 20
  );
  port (
    clk   : in  std_logic;
    rst   : in  std_logic;
    din   : in  std_logic;
    level : out std_logic;
    pulse : out std_logic
  );
end entity;

architecture rtl of debounce is
  constant MAX_COUNT : natural := (CLK_HZ / 1000) * DEBOUNCE_MS;
  signal cnt : natural := 0;
  signal sync1, sync2 : std_logic := '0';
  signal last : std_logic := '0';
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        cnt <= 0;
        sync1 <= '0';
        sync2 <= '0';
        last <= '0';
        pulse <= '0';
        level <= '0';
      else
        pulse <= '0';
        sync1 <= din;
        sync2 <= sync1;
```

```
        if sync2 = level then
          cnt <= 0;
        else
          if cnt = MAX_COUNT then
            level <= sync2;
            cnt <= 0;
          else
            cnt <= cnt + 1;
          end if;
        end if;

        if last = '0' and level = '1' then
          pulse <= '1';
        end if;
        last <= level;
      end if;
    end if;
  end process;
end architecture;
```

4.3 Timekeeper (`timekeeper.vhd`)

Handles time counting and manual adjustment.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity timekeeper is
  port (
    clk       : in  std_logic;
    rst       : in  std_logic;
    tick_1hz  : in  std_logic;
    set_mode  : in  std_logic;
    inc_hr_p  : in  std_logic;
    inc_min_p : in  std_logic;
    sec  : out unsigned(5 downto 0);
    min  : out unsigned(5 downto 0);
    hour : out unsigned(4 downto 0)
  );
end entity;

architecture rtl of timekeeper is
  signal s_sec  : unsigned(5 downto 0) := (others => '0');
  signal s_min  : unsigned(5 downto 0) := (others => '0');
  signal s_hour : unsigned(4 downto 0) := (others => '0');
begin
  sec  <= s_sec;
  min  <= s_min;
  hour <= s_hour;

  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        s_sec <= (others => '0');
        s_min <= (others => '0');
        s_hour <= (others => '0');
      elsif set_mode = '1' then
```

```
        if inc_min_p = '1' then
          s_min <= (s_min = 59) ? (others => '0') : s_min + 1;
        end if;
        if inc_hr_p = '1' then
          s_hour <= (s_hour = 23) ? (others => '0') : s_hour + 1;
        end if;
      elsif tick_1hz = '1' then
        if s_sec = 59 then
          s_sec <= (others => '0');
          if s_min = 59 then
            s_min <= (others => '0');
            s_hour <= (s_hour = 23) ? (others => '0') : s_hour + 1;
          else
            s_min <= s_min + 1;
          end if;
        else
          s_sec <= s_sec + 1;
        end if;
      end if;
    end if;
  end process;
end architecture;
```

4.4 Binary-to-BCD Converter (`bin_to_bcd2.vhd`)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity bin_to_bcd2 is
  port (
    bin  : in  unsigned(6 downto 0);
    tens : out unsigned(3 downto 0);
    ones : out unsigned(3 downto 0)
  );
end entity;

architecture rtl of bin_to_bcd2 is
begin
  process(bin)
    variable v : integer;
  begin
    v := to_integer(bin);
    tens <= to_unsigned(v / 10, 4);
    ones <= to_unsigned(v mod 10, 4);
  end process;
end architecture;
```

4.5 Seven-Segment Display Driver (`sevenseg_mux8.vhd`)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity sevenseg_mux8 is
  port (
    clk      : in  std_logic;
    rst      : in  std_logic;
```

```
    tick_scan: in  std_logic;

    d0 : in unsigned(3 downto 0);
    d1 : in unsigned(3 downto 0);
    d2 : in unsigned(3 downto 0);
    d3 : in unsigned(3 downto 0);
    d4 : in unsigned(3 downto 0);
    d5 : in unsigned(3 downto 0);
    d6 : in unsigned(3 downto 0);
    d7 : in unsigned(3 downto 0);

    -- dp_mask bit = 1 means DP ON for that digit
    dp_mask : in std_logic_vector(7 downto 0);

    an  : out std_logic_vector(7 downto 0);
    seg : out std_logic_vector(6 downto 0);
    dp  : out std_logic
  );
end entity;
architecture rtl of sevenseg_mux8 is
  signal idx : unsigned(2 downto 0) := (others => '0');
  signal cur : unsigned(3 downto 0) := (others => '0');

  -- active-low outputs
  signal an_i  : std_logic_vector(7 downto 0) := (others => '1');
  signal seg_i : std_logic_vector(6 downto 0) := (others => '1');
  signal dp_i  : std_logic := '1';

function bcd_to_7seg_active_low(b : unsigned(3 downto 0)) return
std_logic_vector is
  -- returns seg(6 downto 0) ACTIVE-LOW but ordered as:
  -- seg(0)=a, seg(1)=b, seg(2)=c, seg(3)=d, seg(4)=e, seg(5)=f,
seg(6)=g
  variable r : std_logic_vector(6 downto 0);
begin
  case to_integer(b) is
    when 0 => r := "1000000";
    when 1 => r := "1111001";
    when 2 => r := "0100100";
    when 3 => r := "0110000";
    when 4 => r := "0011001";
    when 5 => r := "0010010";
    when 6 => r := "0000010";
    when 7 => r := "1111000";
    when 8 => r := "0000000";
    when 9 => r := "0010000";
    when others => r := "1111111"; -- blank
  end case;
  return r;
end function;



begin
  an  <= an_i;
  seg <= seg_i;
```

```
  dp  <= dp_i;

  -- scan index
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        idx <= (others => '0');
      else
        if tick_scan = '1' then
          idx <= idx + 1;
        end if;
      end if;
    end if;
  end process;

  -- choose digit + drive outputs
  process(idx, d0,d1,d2,d3,d4,d5,d6,d7, dp_mask)
    variable a : std_logic_vector(7 downto 0);
    variable dm : std_logic;
  begin
    a := (others => '1');
    dm := '0';

    case to_integer(idx) is
      when 0 => cur <= d0; a(0) := '0'; dm := dp_mask(0);
      when 1 => cur <= d1; a(1) := '0'; dm := dp_mask(1);
      when 2 => cur <= d2; a(2) := '0'; dm := dp_mask(2);
      when 3 => cur <= d3; a(3) := '0'; dm := dp_mask(3);
      when 4 => cur <= d4; a(4) := '0'; dm := dp_mask(4);
      when 5 => cur <= d5; a(5) := '0'; dm := dp_mask(5);
      when 6 => cur <= d6; a(6) := '0'; dm := dp_mask(6);
      when others => cur <= d7; a(7) := '0'; dm := dp_mask(7);
    end case;

    an_i  <= a;
    seg_i <= bcd_to_7seg_active_low(cur);

    -- dp_mask bit=1 => dp ON, but dp is active-low
    if dm = '1' then
      dp_i <= '0';
    else
      dp_i <= '1';
    end if;
  end process;

end architecture;
```

4.6 Top-Level Module (`top_clock_nexys_a7.vhd`)

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity top_clock_nexys_a7 is
  port (
    CLK100MHZ : in  std_logic;
```

```
    SW  : in  std_logic_vector(15 downto 0);
    BTNC : in std_logic;
    BTNU : in std_logic;
    BTND : in std_logic;
    AN  : out std_logic_vector(7 downto 0);
    SEG : out std_logic_vector(6 downto 0);
    DP  : out std_logic
  );
end entity;
architecture rtl of top_clock_nexys_a7 is
  signal rst : std_logic;
  signal tick_1hz  : std_logic;
  signal tick_scan : std_logic;
  -- debounced pulses
  signal set_mode : std_logic;
  signal u_level, u_pulse : std_logic;
  signal d_level, d_pulse : std_logic;

  signal sec  : unsigned(5 downto 0);
  signal min  : unsigned(5 downto 0);
  signal hour : unsigned(4 downto 0);

  -- BCD digits
  signal ht, ho : unsigned(3 downto 0);
  signal mt, mo : unsigned(3 downto 0);
  signal st, so : unsigned(3 downto 0);

  signal dp_mask : std_logic_vector(7 downto 0) := (others => '0');
begin
  -- Center button reset (active high)
  rst <= BTNC;

  set_mode <= SW(0);

  u_db: entity work.debounce
    generic map (CLK_HZ => 100_000_000, DEBOUNCE_MS => 20)
    port map (clk => CLK100MHZ, rst => rst, din => BTNU, level => u_level,
pulse => u_pulse);

  d_db: entity work.debounce
    generic map (CLK_HZ => 100_000_000, DEBOUNCE_MS => 20)
    port map (clk => CLK100MHZ, rst => rst, din => BTND, level => d_level,
pulse => d_pulse);

  gen: entity work.clk_en_gen
    generic map (CLK_HZ => 100_000_000, TICK_1HZ_HZ => 1, SCAN_HZ => 1000)
    port map (clk => CLK100MHZ, rst => rst, tick_1hz => tick_1hz, tick_scan
=> tick_scan);

  tk: entity work.timekeeper
    port map (
      clk       => CLK100MHZ,
      rst       => rst,
      tick_1hz  => tick_1hz,
      set_mode  => set_mode,
      inc_min_p => d_pulse,
      inc_hr_p  => u_pulse,
      sec  => sec,
      min  => min,
      hour => hour
    );
```

```
  bcd_h: entity work.bin_to_bcd2
    port map (bin => resize(hour, 7), tens => ht, ones => ho);

  bcd_m: entity work.bin_to_bcd2
    port map (bin => resize(min, 7), tens => mt, ones => mo);

  bcd_s: entity work.bin_to_bcd2
    port map (bin => resize(sec, 7), tens => st, ones => so);

  -- Optional decimal points as separators:
  -- Put DP after HH and after MM (looks like HH.MM.SS on some boards)
  -- Digits mapped below: AN0=rightmost
  --   AN0=sec ones, AN1=sec tens, AN2=min ones, AN3=min tens, AN4=hr ones,
AN5=hr tens
  dp_mask(2) <= '1'; -- DP on min ones digit
  dp_mask(4) <= '1'; -- DP on hr ones digit

  disp: entity work.sevenseg_mux8
    port map (
      clk       => CLK100MHZ,
      rst       => rst,
      tick_scan => tick_scan,

      -- rightmost digits: HHMMSS on AN5..AN0
      d0 => so,  -- AN0
      d1 => st,  -- AN1
      d2 => mo,  -- AN2
      d3 => mt,  -- AN3
      d4 => ho,  -- AN4
      d5 => ht,  -- AN5
      d6 => to_unsigned(15,4), -- blank
      d7 => to_unsigned(15,4), -- blank

      dp_mask => dp_mask,

      an  => AN,
      seg => SEG,
      dp  => DP
    );

end architecture;
```

5. CONSTRAINTS FILE (`nexys_a7_clock.xdc`)

```
## ---- Clock (100 MHz)
set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports {
CLK100MHZ }]
create_clock -add -name sys_clk_pin -period 10.000 -waveform {0 5}
[get_ports { CLK100MHZ }]

## ---- Switches SW[15:0]
set_property -dict { PACKAGE_PIN J15 IOSTANDARD LVCMOS33 } [get_ports {
SW[0] }]
set_property -dict { PACKAGE_PIN L16 IOSTANDARD LVCMOS33 } [get_ports {
SW[1] }]
set_property -dict { PACKAGE_PIN M13 IOSTANDARD LVCMOS33 } [get_ports {
SW[2] }]
```

```
set_property -dict { PACKAGE_PIN R15 IOSTANDARD LVCMOS33 } [get_ports {
SW[3] }]
set_property -dict { PACKAGE_PIN R17 IOSTANDARD LVCMOS33 } [get_ports {
SW[4] }]
set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports {
SW[5] }]
set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports {
SW[6] }]
set_property -dict { PACKAGE_PIN R13 IOSTANDARD LVCMOS33 } [get_ports {
SW[7] }]
set_property -dict { PACKAGE_PIN T8  IOSTANDARD LVCMOS18 } [get_ports {
SW[8] }]
set_property -dict { PACKAGE_PIN U8  IOSTANDARD LVCMOS18 } [get_ports {
SW[9] }]
set_property -dict { PACKAGE_PIN R16 IOSTANDARD LVCMOS33 } [get_ports {
SW[10] }]
set_property -dict { PACKAGE_PIN T13 IOSTANDARD LVCMOS33 } [get_ports {
SW[11] }]
set_property -dict { PACKAGE_PIN H6  IOSTANDARD LVCMOS33 } [get_ports {
SW[12] }]
set_property -dict { PACKAGE_PIN U12 IOSTANDARD LVCMOS33 } [get_ports {
SW[13] }]
set_property -dict { PACKAGE_PIN U11 IOSTANDARD LVCMOS33 } [get_ports {
SW[14] }]
set_property -dict { PACKAGE_PIN V10 IOSTANDARD LVCMOS33 } [get_ports {
SW[15] }]

##Push Buttons
set_property -dict { PACKAGE_PIN N17 IOSTANDARD LVCMOS33 } [get_ports {
BTNC }]
set_property -dict { PACKAGE_PIN M18 IOSTANDARD LVCMOS33 } [get_ports {
BTNU }]
set_property -dict { PACKAGE_PIN P18 IOSTANDARD LVCMOS33 } [get_ports {
BTND }]
set_property -dict { PACKAGE_PIN P17 IOSTANDARD LVCMOS33 } [get_ports {
BTNL }]
set_property -dict { PACKAGE_PIN M17 IOSTANDARD LVCMOS33 } [get_ports {
BTNR }]

##7-seg segments (SEG[0]=CA ... SEG[6]=CG) and DP
set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports {
SEG[0] }]  ;# CA
set_property -dict { PACKAGE_PIN R10 IOSTANDARD LVCMOS33 } [get_ports {
SEG[1] }]  ;# CB
set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports {
SEG[2] }]  ;# CC
set_property -dict { PACKAGE_PIN K13 IOSTANDARD LVCMOS33 } [get_ports {
SEG[3] }]  ;# CD
set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVCMOS33 } [get_ports {
SEG[4] }]  ;# CE
set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports {
SEG[5] }]  ;# CF
set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports {
SEG[6] }]  ;# CG
set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVCMOS33 } [get_ports { DP
}]

##7-seg anodes AN[7:0]
set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports {
AN[0] }]
```

```
set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports {
AN[1] }]
set_property -dict { PACKAGE_PIN T9  IOSTANDARD LVCMOS33 } [get_ports {
AN[2] }]
set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports {
AN[3] }]
set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports {
AN[4] }]
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports {
AN[5] }]
set_property -dict { PACKAGE_PIN K2  IOSTANDARD LVCMOS33 } [get_ports {
AN[6] }]
set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports {
AN[7] }]
```

6. RESULTS

6.1 Functional Verification

The design was synthesized and implemented successfully using Xilinx Vivado 2018.2. After programming the FPGA, the digital clock operated correctly in both normal and set modes.

- Time increments accurately at 1-second intervals.
- Hour and minute adjustments work only when set mode is enabled.
- Display multiplexing is stable and flicker-free.
- Button debounce prevents unintended multiple increments.

6.2 RTL Schematic

The figure below is the RTL schematic generated by Vivado, showing the hierarchy of the clock enable generator, timekeeper, and display driver.

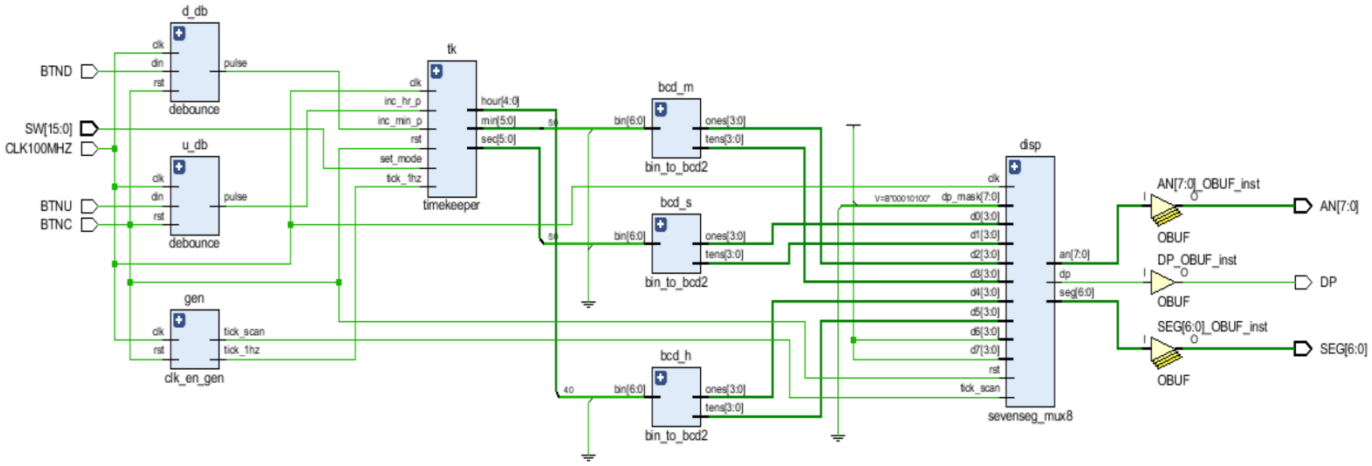


Figure 2: RTL Schematic

6.3 Hardware Output on Nexys A7 Board

The image shows the Nexys A7 FPGA board successfully displaying time in HH:MM:SS format on the seven-segment display, confirming correct operation of the digital clock design.

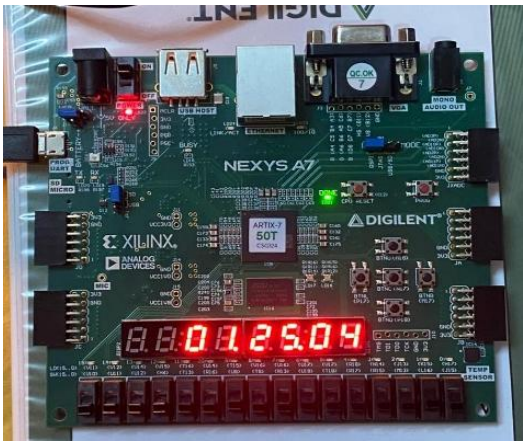


Figure 3: Current time in HH:MM:SS format

6.4 Stopwatch-Style Testing

To verify correct timing behaviour, the clock was compared against a stopwatch:

- The seconds increment matched real-time counting.
- Minute rollover occurred exactly after 60 seconds.



Figure 4: Stopwatch comparison test

7. CONCLUSION

This project successfully demonstrated the design and implementation of a digital clock using VHDL on an FPGA platform. By modularizing the system into clock generation, timekeeping, and display control, the design remained clear and extensible. The project reinforced key concepts such as clock division, synchronous counters, and hardware-based user interfaces.

While the clock does not retain time during power loss, it fulfills all functional requirements of a basic FPGA-based digital clock and provides a strong foundation for future enhancements such as real-time clock integration or alarm features.

8. FUTURE IMPROVEMENTS

Possible extensions of this project include:

- Implementing decrement controls for hours and minutes.
- Adding a stopwatch or alarm mode.

REFERENCES

1. Digilent Inc. (2017). *Nexys A7 FPGA Board Reference Manual*. Digilent. https://digilent.com/reference/programmable-logic/nexys-a7/start
2. Digilent Inc. (2017). *Nexys A7 Master Constraints File and Port Descriptions*. Digilent. https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual
3. Brown, S. D., & Vranesic, Z. G. (2014). *Fundamentals of Digital Logic with VHDL Design*. McGraw-Hill Education.